Introdução a Complexidade e Algoritmos básicos

Escaneie para acessar o slide da Aula 2

luisenilton.github.io/Curso-Maratona/aula2

Sumário da Aula

1. Introdução a Complexidade e algoritmos básicos
2. Sumário da Aula
3. Busca Binária
4. Exemplo (Adivinhe o numero)
5. Quantidade de chutes
6. Problema
7. Por que estudar complexidade?
8. Notação Big O
9. Aplicação na Maratona
10. Analisando código
11. Analisando código
12. Exercicio Motivador
13. Primeira Abordagem
14. Segunda abordagem
15. Observação
16. Divisores de $N = 36$
17. Terceira abordagem
18. Problema
19. Solução
20. Hora do Quiz
21. Referências
22. Contatos
23. Obrigado por acompanhar a aula

Busca Binária

Imagine buscar a palavra **"Laranja"** em um dicionário.

- Você não lê página por página desde a letra A.
- Você abre o livro no meio (ex: letra M).
- Como "L" vem antes de "M", você **descarta toda a segunda metade** do livro.
- Repete o processo na primeira metade até achar

Exemplo (Adivinhe o numero)

Estou pensando em um número de **1 a 100**. A cada chute, eu digo: Maior, Menor ou Acertou.

Exemplo (Adivinhe o numero)

Estou pensando em um número de **1 a 100**. A cada chute, eu digo: Maior, Menor ou Acertou.

- Abordagem 1: Pesquisa Simples (Linear)
  - Chuta 1, depois 2, depois 3...
  - Pior caso: 100 tentativas.

Exemplo (Adivinhe o numero)

Estou pensando em um número de **1 a 100**. A cada chute, eu digo: Maior, Menor ou Acertou.

- Abordagem 1: Pesquisa Simples (Linear)
  - Chuta 1, depois 2, depois 3...
  - Pior caso: 100 tentativas.
- Abordagem 2: Busca Binária
  - Chuta o meio: **50**. *(Eu digo: "Maior")*
  - Metade das opções foram eliminadas. O número está entre 51 e 100.
  - Novo meio: **75**. *(Eu digo: "Menor")*
  - O número com certeza está entre 51 e 74.

Exemplo (Adivinhe o numero)

Estou pensando em um número de **1 a 100**. A cada chute, eu digo: Maior, Menor ou Acertou.

- Abordagem 1: Pesquisa Simples (Linear)
  - Chuta 1, depois 2, depois 3...
  - Pior caso: 100 tentativas.
- Abordagem 2: Busca Binária
  - Chuta o meio: **50**. *(Eu digo: "Maior")*
  - Metade das opções foram eliminadas. O número está entre 51 e 100.
  - Novo meio: **75**. *(Eu digo: "Menor")*
  - O número com certeza está entre 51 e 74.

Quantidade de chutes

Na busca simples, vamos subtraindo as possibilidades de 1 em 1. Na busca binária, dividimos por 2 a quantidade de possibilidades a cada passo. O número máximo de tentativas é quantas vezes podemos dividir N por 2 até chegar a 1.

Quantidade de chutes

Na busca simples, vamos subtraindo as possibilidades de 1 em 1. Na busca binária, dividimos por 2 a quantidade de possibilidades a cada passo. O número máximo de tentativas é quantas vezes podemos dividir N por 2 até chegar a 1.

Isso é a definição de **Logaritmo na base 2**.

Quantidade de chutes

Na busca simples, vamos subtraindo as possibilidades de 1 em 1. Na busca binária, dividimos por 2 a quantidade de possibilidades a cada passo. O número máximo de tentativas é quantas vezes podemos dividir N por 2 até chegar a 1.

Isso é a definição de **Logaritmo na base 2**.

Tamanho de N	Busca Simples $O(N)$	Busca Binária $O(\log N)$
100 elementos	100 chutes	~ 7 chutes
1.000 elementos	1.000 chutes	~ 10 chutes
10^6 elementos	1.000.000 chutes	~ 20 chutes
10^9 elementos	1.000.000.000 chutes	~ 30 chutes

Problema

Dado um array com N elementos responda Q perguntas do tipo "Qual o indice do elemento V no array, se V não estiver no array imprima 'X' ".

Link Problema

Por que estudar complexidade?

Nos problemas de maratona não nos importamos apenas se o algoritmo está correto, mas também com a velocidade que ele executa.

Essa motivação nos leva a estudar complexidade, que nos dá uma estimativa, com base na entrada do tempo que o algoritmo demora para rodar .

Motivos:

Por que estudar complexidade?

Nos problemas de maratona não nos importamos apenas se o algoritmo está correto, mas também com a velocidade que ele executa.

Essa motivação nos leva a estudar complexidade, que nos dá uma estimativa, com base na entrada do tempo que o algoritmo demora para rodar .

Motivos:

- Estimar o tempo de execução sem ter o código.

Por que estudar complexidade?

Nos problemas de maratona não nos importamos apenas se o algoritmo está correto, mas também com a velocidade que ele executa.

Essa motivação nos leva a estudar complexidade, que nos dá uma estimativa, com base na entrada do tempo que o algoritmo demora para rodar .

Motivos:

- Estimar o tempo de execução sem ter o código.
- Avaliar o pior caso para a solução proposta, em competições, os testes dos juizes são feitos para quebrar soluções ineficientes.

Por que estudar complexidade?

Nos problemas de maratona não nos importamos apenas se o algoritmo está correto, mas também com a velocidade que ele executa.

Essa motivação nos leva a estudar complexidade, que nos dá uma estimativa, com base na entrada do tempo que o algoritmo demora para rodar .

Motivos:

- Estimar o tempo de execução sem ter o código.
- Avaliar o pior caso para a solução proposta, em competições, os testes dos juizes são feitos para quebrar soluções ineficientes.
- Saber que estrutura de dados eu posso utilizar, sem estourar o limite de tempo.

Notação Big O

O Big O (O) descreve como o número de operações do seu algoritmo cresce à medida que a entrada (N) aumenta.

Definições

Notação Big O

O Big O (O) descreve como o número de operações do seu algoritmo cresce à medida que a entrada (N) aumenta.

Definições

- **Não mede o tempo em segundos:** Segundos dependem do hardware, da linguagem e do compilador.

Notação Big O

O Big O (O) descreve como o número de operações do seu algoritmo cresce à medida que a entrada (N) aumenta.

Definições

- **Não mede o tempo em segundos:** Segundos dependem do hardware, da linguagem e do compilador.
- **Mede a ordem de grandeza:** Se a entrada dobrar, o tempo dobra? Quadruplica?

Notação Big O

O Big O (O) descreve como o número de operações do seu algoritmo cresce à medida que a entrada (N) aumenta.

Definições

- **Não mede o tempo em segundos:** Segundos dependem do hardware, da linguagem e do compilador.
- **Mede a ordem de grandeza:** Se a entrada dobrar, o tempo dobra? Quadruplica?
- **Foca no Pior Caso:** Assumimos que o universo conspira contra nós. Se a busca pode ir até o final da lista, assumimos que ela vai até o final.

Notação Big O

O Big O (O) descreve como o número de operações do seu algoritmo cresce à medida que a entrada (N) aumenta.

Definições

- **Não mede o tempo em segundos:** Segundos dependem do hardware, da linguagem e do compilador.
- **Mede a ordem de grandeza:** Se a entrada dobrar, o tempo dobra? Quadruplica?
- **Foca no Pior Caso:** Assumimos que o universo conspira contra nós. Se a busca pode ir até o final da lista, assumimos que ela vai até o final.
- **Ignora Constantes:** Para o Big O, $O(2N)$ e $O(50N)$ são apenas $O(N)$. O que importa é a curva de crescimento.

Aplicação na Maratona

Via de regra, em competições usando C++, em 1 segundo o computador processa **10^8 operações.

Em cada problema é definido o tempo limite máximo e as restrições para o tamanho da entrada.

$N \leq$	$\mathcal{O}$ "máximo"
11	$\mathcal{O}(n!)$
22	$\mathcal{O}(2^n.n)$
100	$\mathcal{O}(n^4)$
400	$\mathcal{O}(n^3)$
2000	$\mathcal{O}(n^2.\log_2(n))$
10^4	$\mathcal{O}(n^2)$
10^5	$\mathcal{O}(n.\log_2{}^2(n))$
10^6	$\mathcal{O}(n.\log_2(n))$
10^8	$\mathcal{O}(n)$
10^{18}	$\mathcal{O}(\log_2(n)), \mathcal{O}(1)$

Analisando código

Loops aninhados

Analisando código

Loops aninhados

```
int cnt = 0;
for(int i = 1; i ≤n;i++){
  for(int j = 1;j ≤m;j++){
    cnt++;
  }
}
```

Analisando código

Loops aninhados

```
int cnt = 0;
for(int i = 1; i ≤n;i++){
  for(int j = 1;j ≤m;j++){
    cnt++;
  }
}
```

- Em loops aninhados em cada voltar do loop exterior, você roda o loop interior completo, então você multiplica a complexidade dos dois.
- O(N * M)

Analisando código

Loops separados

Analisando código

Loops separados

```
int cnt = 0;
for(int i = 1; i ≤n;i++){

}

for(int j = 1;j ≤m;j++){
    cnt++;
  }
```

Analisando código

Loops separados

```
int cnt = 0;
for(int i = 1; i ≤n;i++){

}

for(int j = 1;j ≤m;j++){
    cnt++;
  }
```

- Em loops Separados, calculamos a complexidade separadamente e depois somamos.
- O(N + M)

Exercicio Motivador

Divisores de um número

Exercicio Motivador

Divisores de um número

Dado um numero N, encontre todos os divisores dele.

Exercicio Motivador

Divisores de um número

Dado um numero N, encontre todos os divisores dele.

Restrições: N <= 10^9.

Exercicio Motivador

Divisores de um número

Dado um numero N, encontre todos os divisores dele.

Restrições: N <= 10^9.

Tempo limite: 1 segundo.

Primeira Abordagem

Podemos iterar por todos os numeros <= a N e testar se ele divide N.

```
vector<int> divisores;
for(int i = 1; i ≤ n;i++){
  if((n % i) == 0) divisores.push_back(i);
}
```

Primeira Abordagem

Podemos iterar por todos os numeros <= a N e testar se ele divide N.

```
vector<int> divisores;
for(int i = 1; i ≤ n;i++){
  if((n % i) == 0) divisores.push_back(i);
}
```

Qual a complexidade desse código?

Primeira Abordagem

Podemos iterar por todos os numeros <= a N e testar se ele divide N.

```
vector<int> divisores;
for(int i = 1; i ≤ n;i++){
  if((n % i) == 0) divisores.push_back(i);
}
```

Qual a complexidade desse código?

$O(N)$

Primeira Abordagem

Podemos iterar por todos os numeros <= a N e testar se ele divide N.

```
vector<int> divisores;
for(int i = 1; i ≤ n;i++){
  if((n % i) == 0) divisores.push_back(i);
}
```

Qual a complexidade desse código?

$O(N)$

Efetuariamos 10^9 operações o que é muito lento.

Segunda abordagem

É impossivel que um numero maior do que a metade divida N, logo só precisamos testar até N/2

```
for(int i = 1; i ≤ n/2;i++){
  if((n % i) == 0) divisores.push_back(i);
}
```

Segunda abordagem

É impossivel que um numero maior do que a metade divida N, logo só precisamos testar até N/2

```
for(int i = 1; i ≤ n/2;i++){
  if((n % i) == 0) divisores.push_back(i);
}
```

Qual a complexidade desse código?

Segunda abordagem

É impossivel que um numero maior do que a metade divida N, logo só precisamos testar até N/2

```
for(int i = 1; i ≤ n/2;i++){
  if((n % i) == 0) divisores.push_back(i);
}
```

Qual a complexidade desse código?

$O(N/2) = O(N)$

Segunda abordagem

É impossivel que um numero maior do que a metade divida N, logo só precisamos testar até N/2

```
for(int i = 1; i ≤ n/2;i++){
  if((n % i) == 0) divisores.push_back(i);
}
```

Qual a complexidade desse código?

$O(N/2) = O(N)$

Efetuariamos $5 * 10^8$ operações o que ainda é muito lento.

Observação

Todo número N tem uma quantidade finita de divisores, mas eles não estão distribuidos de forma aleatória.

Eles se organizam em pares ao redor do ponto central: $\sqrt{N}$.

Observação

Todo número N tem uma quantidade finita de divisores, mas eles não estão distribuidos de forma aleatória.

Eles se organizam em pares ao redor do ponto central: $\sqrt{N}$.

$$d \times \left(\frac{N}{d}\right) = N$$

Observação

Todo número N tem uma quantidade finita de divisores, mas eles não estão distribuidos de forma aleatória.

Eles se organizam em pares ao redor do ponto central: $\sqrt{N}$.

$$d \times \left(\frac{N}{d}\right) = N$$

Se d é um divisor, $\frac{N}{d}$ também é. Um "espelha" o outro.

Observação

Todo número N tem uma quantidade finita de divisores, mas eles não estão distribuidos de forma aleatória.

Eles se organizam em pares ao redor do ponto central: $\sqrt{N}$.

$$d \times \left(\frac{N}{d}\right) = N$$

Se d é um divisor, $\frac{N}{d}$ também é. Um "espelha" o outro.

Prova por contradição: Existir dois divisores a e b de N (a/N = b) tal que a > $\sqrt{N}$ e b > $\sqrt{N} \Rightarrow a * b > N$, o que é uma contradição.

Divisores de $N = 36$

PARES ENCONTRADOS

Espelho em $\sqrt{36} = 6$

Divisores de $N = 36$

Espelho em $\sqrt{36} = 6$

`Iteração` $i = 1$

PARES ENCONTRADOS

1 36

Divisores de $N = 36$

Espelho em $\sqrt{36} = 6$

`Iteração` $i = 1$

`Iteração` $i = 2$

PARES ENCONTRADOS

1	36
2	18

Divisores de $N = 36$

Espelho em $\sqrt{36} = 6$

Iteração $i = 1$

Iteração $i = 2$

Iteração $i = 3$

PARES ENCONTRADOS

1	36
2	18
3	12

Divisores de $N = 36$

Espelho em $\sqrt{36} = 6$

Iteração $i = 1$

Iteração $i = 2$

Iteração $i = 3$

Iteração $i = 4$

PARES ENCONTRADOS

1	36
2	18
3	12
4	9

Divisores de $N = 36$

Espelho em $\sqrt{36} = 6$

Iteração $i = 1$

Iteração $i = 2$

Iteração $i = 3$

Iteração $i = 4$

Iteração $i = 5$

Não divide (pula)

PARES ENCONTRADOS

1	36
2	18
3	12
4	9

Divisores de $N = 36$

Espelho em $\sqrt{36} = 6$

Iteração $i = 1$

Iteração $i = 2$

Iteração $i = 3$

Iteração $i = 4$

Iteração $i = 5$

Não divide (pula)

Iteração $i = 6 = \sqrt{36}$

PARES ENCONTRADOS

1	36
2	18
3	12
4	9
6	6

Divisores de $N = 36$

Espelho em $\sqrt{36} = 6$

Iteração $i = 1$

Iteração $i = 2$

Iteração $i = 3$

Iteração $i = 4$

Iteração $i = 5$

Não divide (pula)

Iteração $i = 6 = \sqrt{36}$

PARES ENCONTRADOS

1	36
2	18
3	12
4	9
6	6

Iterações: 6 (vs 36)

Após a raiz, os pares apenas se repetem de trás pra frente.

Terceira abordagem

Com base na observação anterior, só precisamos testar os numeros até $\sqrt{N}$

```
for(int i = 1; i * i ≤ n;i++){
  if((n % i) == 0) {
    divisores.push_back(i);
    if(i ≠ n/i) divisores.push_back(n/i);
    // precisamos tratar o caso da raiz quadrada pra não ficar duplicado
  }
}
```

Terceira abordagem

Com base na observação anterior, só precisamos testar os numeros até $\sqrt{N}$

```
for(int i = 1; i * i ≤ n;i++){
  if((n % i) == 0) {
    divisores.push_back(i);
    if(i ≠ n/i) divisores.push_back(n/i);
    // precisamos tratar o caso da raiz quadrada pra não ficar duplicado
  }
}
```

Qual a complexidade desse código?

Terceira abordagem

Com base na observação anterior, só precisamos testar os numeros até $\sqrt{N}$

```
for(int i = 1; i * i ≤ n;i++){
  if((n % i) == 0) {
    divisores.push_back(i);
    if(i ≠ n/i) divisores.push_back(n/i);
    // precisamos tratar o caso da raiz quadrada pra não ficar duplicado
  }
}
```

Qual a complexidade desse código?

$O(\sqrt{N})$

Terceira abordagem

Com base na observação anterior, só precisamos testar os numeros até $\sqrt{N}$

```
for(int i = 1; i * i ≤ n;i++){
  if((n % i) == 0) {
    divisores.push_back(i);
    if(i ≠ n/i) divisores.push_back(n/i);
    // precisamos tratar o caso da raiz quadrada pra não ficar duplicado
  }
}
```

Qual a complexidade desse código?

$O(\sqrt{N})$

Efetuariamos $31 * 10^4$ operações o que passa com bastante folga no tempo limite.

Problema

Responda Q consultas no formato "O numero N é Primo?", se o numero N for primo imprima "YES", caso contrário imprima "NO".

Q <= 10^2

N <= 10^{10}

Link do Problema

Solução

```
bool eh_primo = true;
for(int i = 1; i * i ≤n;i++){
  if((n % i) == 0) {
    eh_primo = false;
  }
}
```

Hora do Quiz

Peguem o link do quiz no chat.

Referências

- Documentação C++
- Livro Entendendo algoritmos
- Aula UFMG
- Série Harmônica

Contatos

Entre no discord para dúvidas

Entre no grupo de questões

Obrigado por acompanhar a aula

Por favor preencha o formulário de feedback com sugestões e críticas para a próxima aula 🙃.